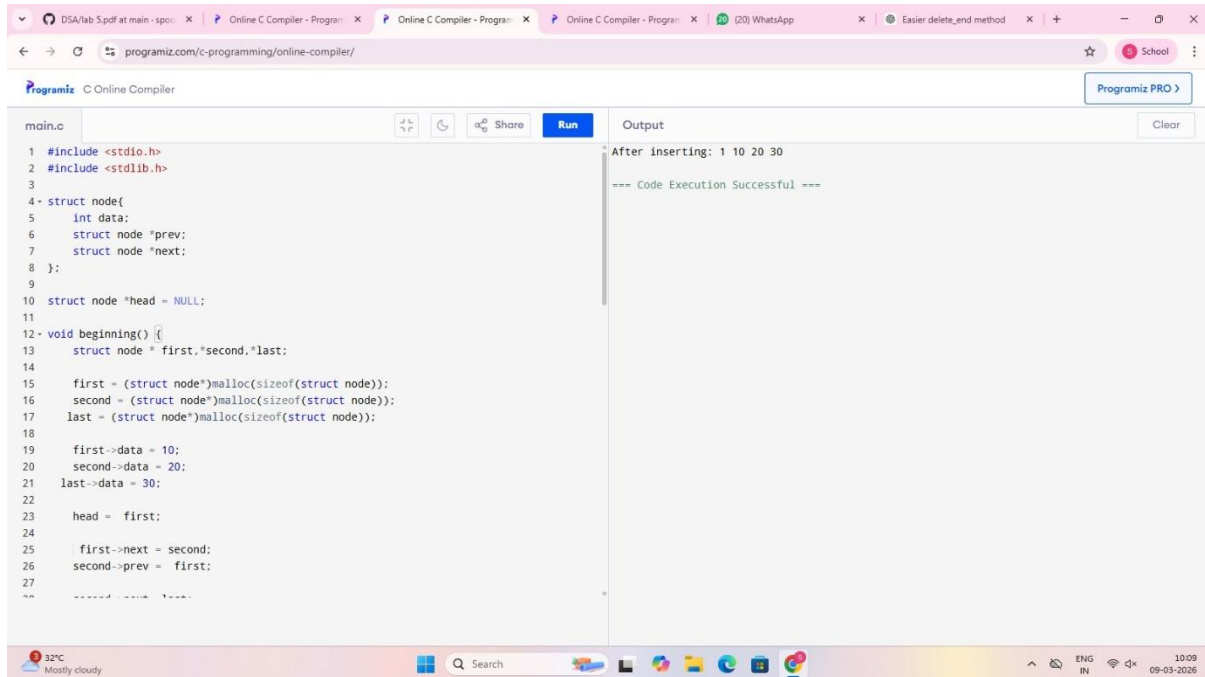


Doubly circular linked list

Insertion at beginning

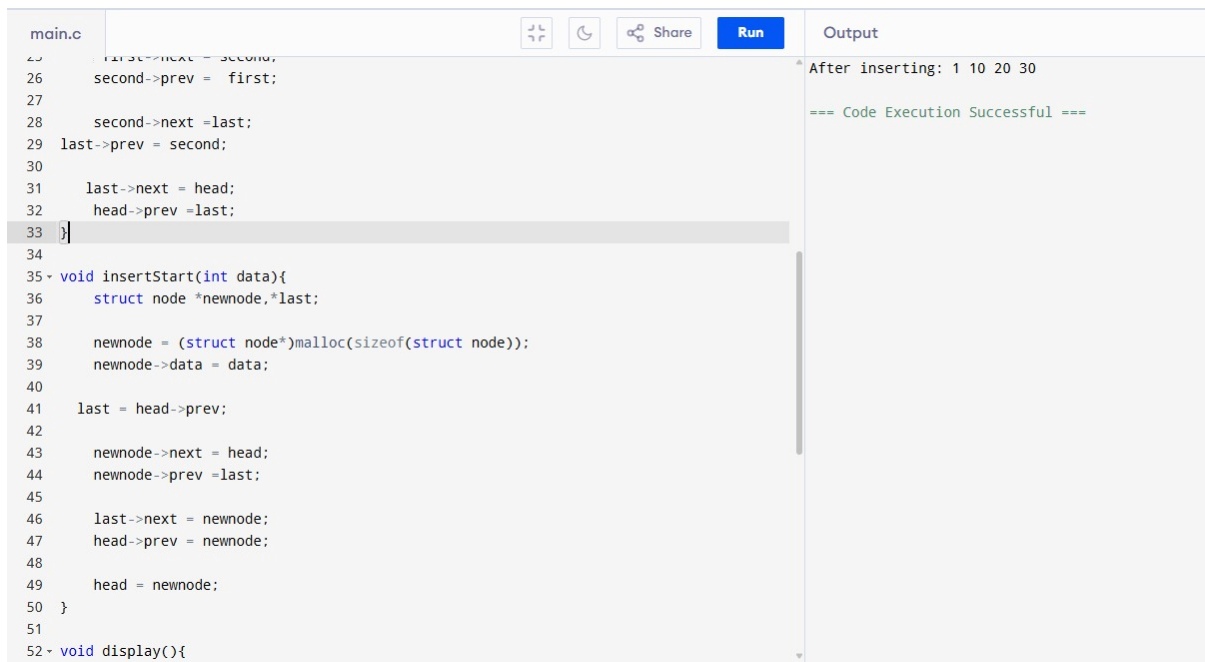


```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *prev;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 void beginning() {
13     struct node *first,*second,*last;
14
15     first = (struct node*)malloc(sizeof(struct node));
16     second = (struct node*)malloc(sizeof(struct node));
17     last = (struct node*)malloc(sizeof(struct node));
18
19     first->data = 10;
20     second->data = 20;
21     last->data = 30;
22
23     head = first;
24
25     first->next = second;
26     second->prev = first;
27
28     // ... more code ...
29 }
```

Output

After inserting: 1 10 20 30

=== Code Execution Successful ===



```
main.c
25     first->next = second;
26     second->prev = first;
27
28     second->next = last;
29     last->prev = second;
30
31     last->next = head;
32     head->prev = last;
33 }
34
35 void insertStart(int data){
36     struct node *newnode,*last;
37
38     newnode = (struct node*)malloc(sizeof(struct node));
39     newnode->data = data;
40
41     last = head->prev;
42
43     newnode->next = head;
44     newnode->prev = last;
45
46     last->next = newnode;
47     head->prev = newnode;
48
49     head = newnode;
50 }
51
52 void display(){
```

Output

After inserting: 1 10 20 30

=== Code Execution Successful ===

main.c

Share

Run

```
52- void display(){
53     struct node *temp = head;
54
55-     if(head == NULL){
56         printf("List is empty");
57         return;
58     }
59
60     printf("%d ", temp->data);
61     temp = temp->next;
62
63-     while(temp != head){
64         printf("%d ", temp->data);
65         temp = temp->next;
66     }
67 }
68
69- int main(){
70     beginning();
71     insertStart(1);
72
73     printf("After inserting: ");
74     display();
75
76     return 0;
77 }
78
```

Output

After inserting: 1 10 20 30
=== Code Execution Successful ===

Insertion at end

main.c

Share

Run

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void beginning()
14 {
15     struct node *first, *second, *last;
16
17     first = (struct node*)malloc(sizeof(struct node));
18     second = (struct node*)malloc(sizeof(struct node));
19     last = (struct node*)malloc(sizeof(struct node));
20
21     first->data = 10;
22     second->data = 20;
23     last->data = 30;
24
25     head = first;
26
```

Output

After inserting at end: 10 20 30 40
=== Code Execution Successful ===

main.c

Share

Run

```
25 head = first;
26
27 first->next = second;
28 second->prev = first;
29
30 second->next = last;
31 last->prev = second;
32
33 last->next = head;
34 head->prev = last;
35 }
36
37 void insertEnd(int data)
38 {
39     struct node *newnode, *last;
40
41     newnode = (struct node*)malloc(sizeof(struct node));
42     newnode->data = data;
43
44     last = head->prev;
45
46     newnode->next = head;
47     newnode->prev = last;
48
49     last->next = newnode;
50     head->prev = newnode;
```

Output

After inserting at end: 10 20 30 40

=== Code Execution Successful ===

main.c

Share

Run

```
50 head->prev = newnode;
51 }
52
53 void display()
54 {
55     struct node *temp = head;
56
57     if(head == NULL)
58     {
59         printf("List is empty");
60         return;
61     }
62
63     printf("%d ", temp->data);
64     temp = temp->next;
65
66     while(temp != head)
67     {
68         printf("%d ", temp->data);
69         temp = temp->next;
70     }
71 }
72
73 int main()
74 {
75     beginning();
```

Output

After inserting at end: 10 20 30 40

=== Code Execution Successful ===

main.c	Output
<pre>58- { 59 printf("List is empty"); 60 return; 61 } 62 63 printf("%d ", temp->data); 64 temp = temp->next; 65 66 while(temp != head) 67- { 68 printf("%d ", temp->data); 69 temp = temp->next; 70 } 71 } 72 73 int main() 74- { 75 beginning(); 76 77 insertEnd(40); 78 79 printf("After inserting at end: "); 80 display(); 81 82 return 0; 83 }</pre>	After inserting at end: 10 20 30 40 === Code Execution Successful ===

Insertion at position

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct node 5- { 6 int data; 7 struct node *prev; 8 struct node *next; 9 }; 10 11 struct node *head = NULL; 12 13 void beginning() 14- { 15 struct node *first,*second,*last; 16 17 first = (struct node*)malloc(sizeof(struct node)); 18 second = (struct node*)malloc(sizeof(struct node)); 19 last = (struct node*)malloc(sizeof(struct node)); 20 21 first->data = 10; 22 second->data = 20; 23 last->data = 30; 24 25 head = first; 26 }</pre>	After inserting at position: 10 15 20 30 === Code Execution Successful ===

main.c

Share

Run

25

head = first;

26

27

first->next = second;

28

second->prev = first;

29

30

second->next = last;

31

last->prev = second;

32

33

last->next = head;

34

head->prev = last;

35

}

36

37

void insertPosition(int data,int pos)

38

{

39

struct node *newnode,*temp;

40

int i;

41

42

newnode = (struct node*)malloc(sizeof(struct node));

43

newnode->data = data;

44

45

temp = head;

46

47

for(i=1;i<pos-1;i++)

48

{

49

temp = temp->next;

main.c

Share

Run

49

temp = temp->next;

50

}

51

52

newnode->next = temp->next;

53

newnode->prev = temp;

54

55

temp->next->prev = newnode;

56

temp->next = newnode;

57

}

58

59

void display()

60

{

61

struct node *temp = head;

62

63

if(head == NULL)

64

{

65

printf("List is empty");

66

return;

67

}

68

69

printf("%d ", temp->data);

70

temp = temp->next;

71

72

while(temp != head)

73

{

74

printf("%d ", temp->data);

After inserting at position: 10 15 20 30

=== Code Execution Successful ===

```
main.c
64 {
65     printf("List is empty");
66     return;
67 }
68
69 printf("%d ", temp->data);
70 temp = temp->next;
71
72 while(temp != head)
73 {
74     printf("%d ", temp->data);
75     temp = temp->next;
76 }
77 }
78
79 int main()
80 {
81     beginning();
82
83     insertPosition(15,2);
84
85     printf("After inserting at position: ");
86     display();
87
88     return 0;
89 }
```

Output

After inserting at position: 10 15 20 30

=== Code Execution Successful ===

Deletion at beginning

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node
5 {
6     int data;
7     struct node *prev;
8     struct node *next;
9 };
10
11 struct node *head = NULL;
12
13 void beginning()
14 {
15     struct node *first,*second,*last;
16
17     first = (struct node*)malloc(sizeof(struct node));
18     second = (struct node*)malloc(sizeof(struct node));
19     last = (struct node*)malloc(sizeof(struct node));
20
21     first->data = 10;
22     second->data = 20;
23     last->data = 30;
24
25     head = first;
26 }
```

Output

After deleting at beginning: 20 30

=== Code Execution Successful ===

main.c	Run	Output
<pre>25 head = first; 26 27 first->next = second; 28 second->prev = first; 29 30 second->next = last; 31 last->prev = second; 32 33 last->next = head; 34 head->prev = last; 35 } 36 37 void deleteBeginning() 38 { 39 struct node *temp,*last; 40 41 if(head == NULL) 42 { 43 printf("List is empty"); 44 return; 45 } 46 47 temp = head; 48 last = head->prev; 49 50 head = head->next;</pre>	Run	After deleting at beginning: 20 30 === Code Execution Successful ===
<pre>46 47 temp = head; 48 last = head->prev; 49 50 head = head->next; 51 head->prev = last; 52 last->next = head; 53 54 free(temp); 55 } 56 57 void display() 58 { 59 struct node *temp = head; 60 61 if(head == NULL) 62 { 63 printf("List is empty"); 64 return; 65 } 66 67 printf("%d ", temp->data); 68 temp = temp->next; 69 70 while(temp != head) 71 {</pre>	Run	After deleting at beginning: 20 30 === Code Execution Successful ===

```
main.c
62- {
63-     printf("List is empty");
64-     return;
65- }
66-
67- printf("%d ", temp->data);
68- temp = temp->next;
69-
70- while(temp != head)
71- {
72-     printf("%d ", temp->data);
73-     temp = temp->next;
74- }
75- }
76-
77- int main()
78- {
79-     beginning();
80-
81-     deleteBeginning();
82-
83-     printf("After deleting at beginning: ");
84-     display();
85-
86-     return 0;
87- }
```

Output

After deleting at beginning: 20 30

=== Code Execution Successful ===

Deletion at End

Programiz
C Online Compiler

Never struggle with DSA again.
Learn with interactive visuals

Try Now

Programiz PRO

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *prev;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 void end(){
13     struct node *first,*last;
14
15     head = (struct node*)malloc(sizeof(struct node));
16     first = (struct node*)malloc(sizeof(struct node));
17     last = (struct node*)malloc(sizeof(struct node));
18
19     head->data = 0;
20     first->data = 1;
21     last->data = 2;
22
23     head->prev = NULL;
24     head->next = first;
25
26     first->prev = head;
27     first->next = last;
28 }
```

Output

Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===

Premium Courses by Programiz

Learn More

32°C Mostly cloudy

Search

ENG IN 10:10 09-03-2026

DSA/lab 5.pdf at main · spoorl · Online C Compiler - Programiz · Online C Compiler - Programiz · Online C Compiler - Programiz · (20) WhatsApp

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Learn Data Structures and Algorithms with live code visualizations

Try Now

Programiz PRO

main.c

```
26 first->prev = head;
27 first->next = last;
28
29 last->prev = first;
30 last->next = NULL;
31 }
32
33 void deleteEnd(){
34     struct node *delNode = head;
35
36     if(head == NULL){
37         printf("List is empty\n");
38         return;
39     }
40
41     while(delNode->next != NULL){
42         delNode = delNode->next;
43     }
44
45     if(delNode->prev != NULL){
46         delNode->prev->next = NULL;
47     }
48     else{
49         head = NULL;
50     }
51
52     free(delNode);
```

Output

Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===

Premium Courses by Programiz

Learn More

32°C Mostly cloudy

Search

ENG IN 10:11 09-03-2026

main.c

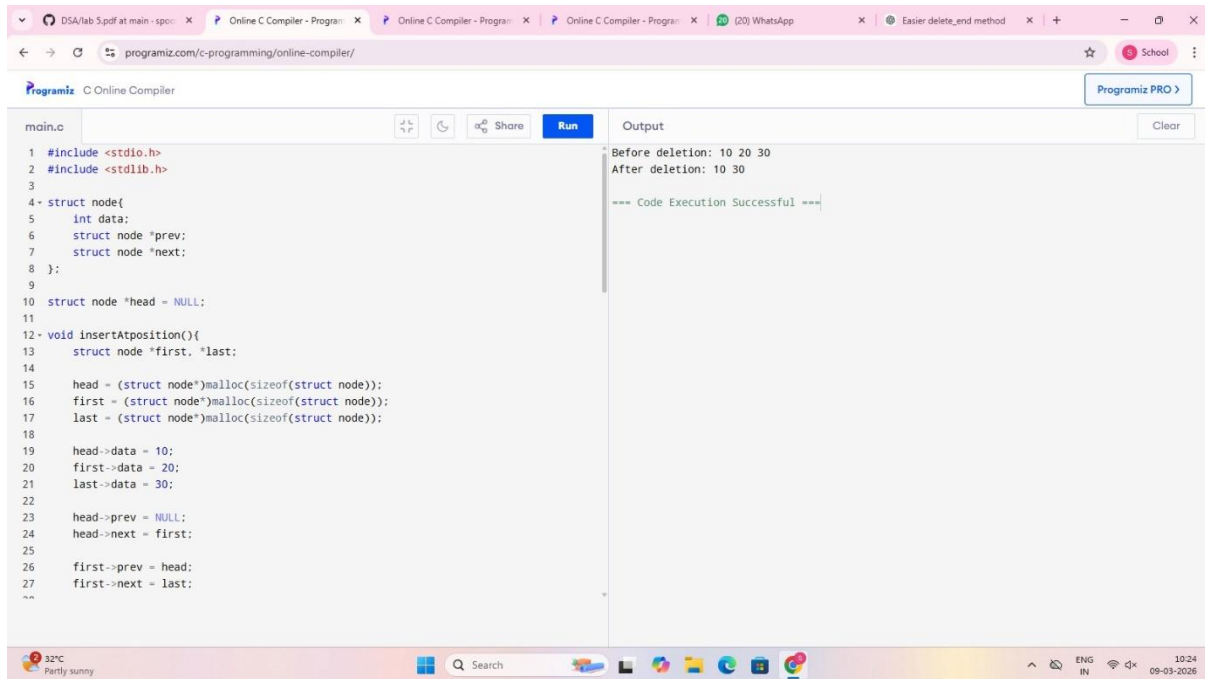
```
50 }
51
52 free(delNode);
53 }
54
55 void display(){
56     struct node *temp = head;
57
58     while(temp != NULL){
59         printf("%d ", temp->data);
60         temp = temp->next;
61     }
62 }
63
64 int main(){
65     end();
66
67     printf("Before deleting: ");
68     display();
69
70     deleteEnd();
71
72     printf("\nAfter deleting: ");
73     display();
74
75     return 0;
76 }
```

Output

Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===

Deletion at position



The screenshot shows a web browser with multiple tabs, including 'DSA/lab 5.pdf at main - spoc...', 'Online C Compiler - Program...', and 'WhatsApp'. The address bar shows 'programiz.com/c-programming/online-compiler/'. The main content area is the 'Programiz C Online Compiler' interface. The code editor on the left contains the following C code:

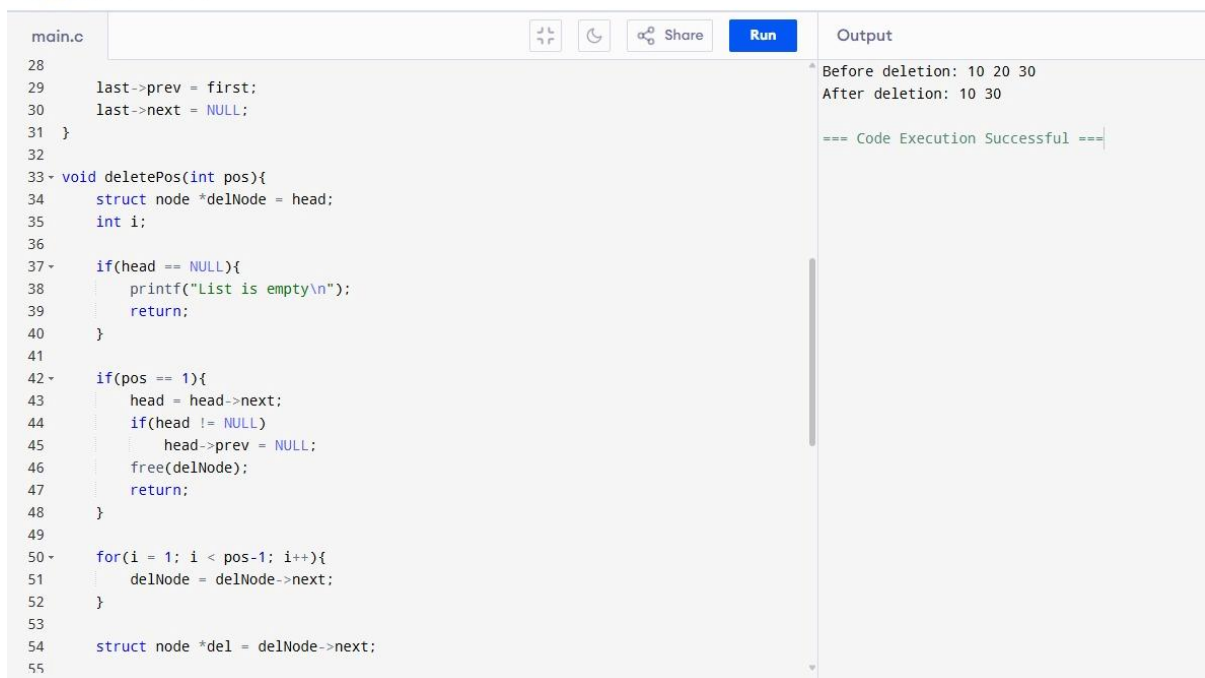
```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *prev;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 void insertAtPosition(){
13     struct node *first, *last;
14
15     head = (struct node*)malloc(sizeof(struct node));
16     first = (struct node*)malloc(sizeof(struct node));
17     last = (struct node*)malloc(sizeof(struct node));
18
19     head->data = 10;
20     first->data = 20;
21     last->data = 30;
22
23     head->prev = NULL;
24     head->next = first;
25
26     first->prev = head;
27     first->next = last;
28 }
```

The output window on the right shows the following text:

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```

The bottom status bar shows a weather icon for '32°C Partly sunny', a search bar, and system icons for language (ENG), region (IN), and date/time (10:24 09-03-2026).



The screenshot shows the same online C compiler interface, but with a different code file named 'main.c'. The code defines a deletePos function. The output window remains the same as in the previous screenshot.

```
main.c
28
29     last->prev = first;
30     last->next = NULL;
31 }
32
33 void deletePos(int pos){
34     struct node *delNode = head;
35     int i;
36
37     if(head == NULL){
38         printf("List is empty\n");
39         return;
40     }
41
42     if(pos == 1){
43         head = head->next;
44         if(head != NULL)
45             head->prev = NULL;
46         free(delNode);
47         return;
48     }
49
50     for(i = 1; i < pos-1; i++){
51         delNode = delNode->next;
52     }
53
54     struct node *del = delNode->next;
55 }
```

The output window on the right shows the following text:

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```

DSA/lab 5.pdf at main · spoc · Online C Compiler · Program · Online C Compiler · Program · Online C Compiler · Program · (20) WhatsApp · Easier delete_end method · + · - ·

programiz.com/c-programming/online-compiler/

Programiz C Online Compiler

main.c

```
45 head->prev = NULL;
46 free(delNode);
47 return;
48 }
49
50 for(i = 1; i < pos-1; i++){
51     delNode = delNode->next;
52 }
53
54 struct node *del = delNode->next;
55
56 delNode->next = del->next;
57
58 if(del->next != NULL)
59     del->next->prev = delNode;
60
61 free(del);
62 }
63
64 void display(){
65     struct node *temp = head;
66
67     while(temp != NULL){
68         printf("%d ", temp->data);
69         temp = temp->next;
70     }
71 }
```

Run

Output

Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===

32°C Partly sunny

Search

ENG IN 10:25 09-03-2026

DSA/lab 5.pdf at main · spoc · Online C Compiler · Program · Online C Compiler · Program · Online C Compiler · Program · (20) WhatsApp · Easier delete_end method · + · - ·

programiz.com/c-programming/online-compiler/

Programiz C Online Compiler

main.c

```
60 free(del);
61 }
62
63
64 void display(){
65     struct node *temp = head;
66
67     while(temp != NULL){
68         printf("%d ", temp->data);
69         temp = temp->next;
70     }
71 }
72
73 int main(){
74     insertAtPosition();
75
76     printf("Before deletion: ");
77     display();
78
79     deletePos(2);
80
81     printf("\nAfter deletion: ");
82     display();
83
84     return 0;
85 }
86 }
```

Run

Output

Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===

32°C Partly sunny

Search

ENG IN 10:25 09-03-2026